

OOP Project Report

Title: Snakes and Ladders
BSCS-2A

Group Members

Name	Enrolment
Mahaad Akbar	02-134241-094
Husnain Bin Saeed	02-134241-012
Muhammad Ahsan	02-134241-101
Abdul Wasay Rais	02-134241-067

Introduction

The Snakes and Ladders game is a digital recreation of the classic board game. This project was developed to provide an engaging and interactive platform for players to enjoy the game. The game supports 2 to 4 players, offering a multiplayer experience with competitive and strategic gameplay. Additionally, it features three unique boards, allowing players to choose their preferred game mode. The project also includes an interactive web-based GUI for an enhanced user experience.

Features

1. Multiplayer Support: The game can be played by 2 to 4 players, catering to diverse group sizes.
2. Multiple Board Options: Three different board designs are available, providing variety and personalization for the players.
3. Player Statistics: Win statistics for players are stored in a PostgreSQL database, allowing for detailed tracking and analysis.
4. Interactive UI: The frontend is designed with JavaScript, HTML, and CSS, offering a visually appealing and intuitive user interface.
5. Real-Time Updates: The game provides instant feedback on player movements and interactions, ensuring a smooth gaming experience.

Technologies Used

1. Backend Development:

The backend logic of the game, including player turns, dice rolls, and game rules, was implemented using Java. Java's object-oriented features facilitated the development of a modular and scalable codebase.

2. Database:

PostgreSQL was used as the database management system to store player statistics, including win records and game history. This ensures data persistence and enables the analysis of player performance.

3. Frontend Development:

JavaScript, HTML, and CSS were used to create an interactive and responsive user interface. These technologies allowed for dynamic updates, animations, and user-friendly navigation.

4. Integration:

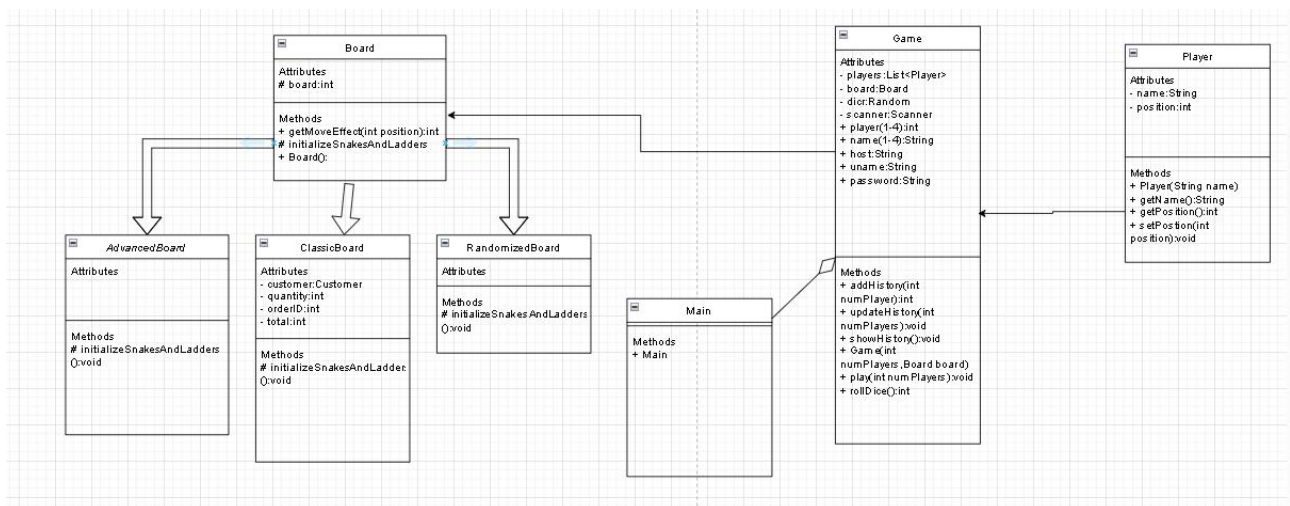
The backend and frontend were integrated, enabling real-time communication between the game logic and user interface. APIs were used for data retrieval and updates.

5. Testing and Debugging:

Comprehensive testing was conducted to ensure the correctness of game logic, database interactions, and UI responsiveness. Bugs were identified and resolved during iterative testing phases.

Methodology

UML Diagram



The game is built using object-oriented programming principles and includes abstraction, encapsulation, and polymorphism. The following is an outline of each class and its members.

1. Player Class

Represents a player in the game.

- Members:
 - String name: Stores the name of the player.
 - int position: Tracks the player's position on the board.
- Methods:
 - Player(String name): Constructor to initialize the player's name and set the position to 0.
 - String getName(): Returns the name of the player.
 - int getPosition(): Returns the current position of the player.
 - void setPosition(int position): Updates the player's position.

2. Abstract Board Class

An abstract class that Defines the structure of a board and ensures initialization of snakes and ladders, it has 3 derived classes that represent different board

- Members:
 - int board: A 2 dimensional 10 x10 array representing the board.
- Methods:
 - public Board(): Initializes the board array and calls the abstract method to set up snakes and ladders.
 - void initializeSnakesAndLadders(): Abstract method for setting up the board, implemented by subclasses.
 - int getMoveEffect(): Returns the effect (snake or ladder) at a given board position.

3. Game Class

Manages the gameplay, player turns, and interactions with the board.

- Members:
 - List<Player> players: stores the list of players.
 - Board board: The board used for the game, an object of the board class.
 - Random dice: A random number generator for dice rolls using the Random class in Java.
 - Scanner scanner: Handles user input for interaction.
- Methods:
 - public Game(int numPlayers, Board board): Constructor to initialize the game with a specified number of players and a chosen board.
 - public void play(): Contains the main game loop, prompting players to roll the dice and updating their positions based on board effects.

- `private int rollDice()`: Simulates a dice roll, returning a random number between 1 and 6.

Conclusion

This project highlights the collaborative use of multiple technologies to deliver a functional and enjoyable gaming application. Focus on code modularity using OOP principles allows more features to be added to the game easily.